RouteGrid v1 — End-to-End Implementation Plan

Project Summary

RouteGrid is a **real-time urban logistics optimization and delivery management platform** — a modular monolith built with **FastAPI + PostgreSQL/PostGIS + Redis + OR-Tools** on the backend and **React + TypeScript + Vite + Leaflet** on the frontend.

The core flow: **Order intake → Warehouse selection → Inventory reservation → Batch vehicle assignment & route optimization (OR-Tools) → Simulated GPS tracking → Disruption recovery → Delivery completion**.

Current State Assessment

✅ What's Done (and Done Right)

Item	Status	Verdict
Project folder structure (`backend/` , `frontend/` , `docs/` , `data/` , `scripts/` , `infra/`)	✅ Complete	Correct — matches the architecture doc
Backend module skeleton (`api/` , `core/` , `db/` , `events/` , `models/` , `optimization/` , `repositories/` , `schemas/` , `services/` , `simulation/`)	✅ Scaffold	Correct — all modules from architecture.md are present
Frontend module skeleton (`components/` , `hooks/` , `layouts/` , `lib/` , `pages/` , `services/` , `types/` , `utils/`)	✅ Scaffold	Correct — matches architecture.md section 5
	✅ Complete	Excellent — 54 functional requirements, NFRs, scope boundaries, open questions
	✅ Complete	Excellent — ER diagram, sequence diagrams, module descriptions, clear rationale
	✅ Complete	Correct
Git history (4 commits)	✅ Clean	Good progression

⚠️ Issues Found in Existing Code (Must Fix)

1. — Unused import and deprecated parameter

```
python
# Line 12: `Optional` is imported but never used
from typing import Optional  # ← REMOVE

# Line 18: `future=True` is a SQLAlchemy 1.4 bridge parameter.
# In SQLAlchemy 2.0+, it's ignored/deprecated. If targeting 2.0, remove it.
# If targeting 1.4, it's fine but the docstring should clarify.
```

Fix: Remove unused `Optional` import. Decide on SQLAlchemy version — if 2.0+, drop the `future` parameter.

2. — Not FastAPI-compatible

The `get_session` is a synchronous context manager. FastAPI uses **async dependency injection** with `Depends()`. This will need to be replaced with:

```
python
# What's needed for FastAPI:
async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with async_session_factory() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

Fix: The current `get_session` is okay as a utility for scripts/migrations, but the project will need an **async** session dependency for FastAPI. This will be built properly in Phase 1.

3. Schema DDL — Missing columns

The DDL in has gaps:

Table	Missing Column	Why It's Needed
`order`	`delivery location geography(Point, 4326)`	Where does the customer want delivery? Without this, routing can't calculate distances
`vehicle`	`depot warehouse id UUID REFERENCES warehouse(id)`	Which warehouse is the vehicle based at? Required for route start point
`route`	`started at timestamptz`, `completed_at timestamptz`	Need to know when a route began/ended for telemetry and audit
`route_stop`	`status text NOT NULL DEFAULT 'PENDING'`	Each stop needs its own lifecycle (PENDING → ARRIVED → COMPLETED)
`route_stop`	`arrived at timestamptz`, `completed_at timestamptz`	Timestamps for when the vehicle reached and completed each stop
`fulfillment_group`	`delivery location geography(Point, 4326)`	The delivery destination for this specific fulfillment group

Fix: Will be addressed when we write the actual SQLAlchemy models and Alembic migrations in Phase 1.

4. No dependency management files

There is **no** `requirements.txt`, `pyproject.toml`, or `package.json` anywhere in the project. The `.venv` exists but has no declared dependencies.

Fix: Create `pyproject.toml` for backend, `package.json` for frontend in Phase 1.

Implementation Plan — 10 Phases

> IMPORTANT
>
> Each phase produces **runnable, testable code** with clear commands you can execute. Phases are sequential — each builds on the previous. Every phase ends with a verification step.

Phase 1: Project Foundation & Database Layer

Goal: Working FastAPI app that starts, connects to PostgreSQL/PostGIS, runs Alembic migrations, and has all 13 tables created.

What you'll understand: How the app boots, how SQLAlchemy models map to database tables, how Alembic manages schema changes, and how the async session dependency works in FastAPI.

Tasks

1. **Create** `backend/pyproject.toml` — Declare all Python dependencies:
   - `fastapi`, `uvicorn[standard]`, `sqlalchemy[asyncio]`, `asyncpg`, `alembic`
   - `geoalchemy2`, `pydantic`, `pydantic-settings`, `redis[hiredis]`
   - `ortools`, `shapely`, `httpx` (for testing)
   - `pytest`, `pytest-asyncio`, `pytest-cov` (dev deps)
2. **Create** `backend/app/core/config.py` — Pydantic Settings class that reads from env vars:
   - `DATABASE_URL`, `REDIS_URL`, `BATCH_SIZE`, `BATCH_TIMEOUT_SECONDS`, `SPEED_FACTOR_KMH`
   - Validates and provides typed access to all configuration
3. **Create** `backend/app/core/auth.py` — The `get_current_user` dependency seam:
   - Returns a hardcoded fake user (`User(id=UUID, name="Dev Operator", role="dispatcher")`)
   - Threaded through all route handlers via `Depends(get_current_user)`

4. **Rewrite** `backend/app/db/` — Async database layer:
    - `engine.py` → Create async engine using `create_async_engine` + `asyncpg`
    - `session.py` → `AsyncSession` factory + FastAPI dependency `get_db()`
    - `base.py` → Declarative base with common columns (id, created_at, updated_at)
5. **Create all 13 SQLAlchemy models** in `backend/app/models/`:
    - `order.py` — Order, OrderItem
    - `fulfillment.py` — FulfillmentGroup, FulfillmentGroupItem
    - `warehouse.py` — Warehouse, Inventory
    - `reservation.py` — Reservation
    - `fleet.py` — Vehicle, Driver
    - `route.py` — Route, RouteStop
    - `telemetry.py` — TelemetrySnapshot
    - `outbox.py` — OutboxEvent
6. **Initialize Alembic** and create initial migration with all tables + PostGIS extension
7. **Create** `backend/app/main.py` — FastAPI app factory with:
    - Lifespan handler (startup: create engine, connect Redis; shutdown: dispose)
    - CORS middleware for frontend
    - Health check endpoint (`GET /health`)
8. **Create** `.env` **file** with local dev database URL
9. **Create** `docker-compose.yml` for PostgreSQL + PostGIS and Redis containers

Verification

```
bash
# Start infrastructure
docker-compose up -d

# Install dependencies
cd backend && pip install -e ".[dev]"

# Run migrations
alembic upgrade head

# Start the app
uvicorn app.main:app --reload

# Test health endpoint
curl http://localhost:8000/health
# Expected: {"status": "ok", "database": "connected", "redis": "connected"}
```

Phase 2: Seed Data & Core CRUD APIs

Goal: Populate the database with a realistic synthetic dataset for a real city (e.g., Hyderabad bounding box) and expose CRUD endpoints for warehouses, vehicles, drivers, and products (SKUs).

What you'll understand: How seed data scripts work, how Pydantic schemas validate API input/output, how the repository pattern separates DB queries from business logic.

Tasks

1. **Create seed data script** `scripts/seed_data.py`:
    - 5 warehouses with real Hyderabad coordinates
    - 40 SKUs with realistic names, weights, and per-warehouse inventory counts
    - 25 vehicles with weight capacities (500–1500 kg) assigned to warehouses
    - 25 drivers, each assigned to one vehicle
    - 50 customer locations within the Hyderabad bounding box
2. **Create Pydantic schemas** in `backend/app/schemas/`:
    - `warehouse.py` — WarehouseCreate, WarehouseRead, WarehouseWithInventory
    - `fleet.py` — VehicleRead, VehicleCreate, DriverRead
    - `inventory.py` — InventoryRead, InventoryUpdate

- `common.py` — Pagination, ErrorResponse, GeoPoint

3. **Create repositories** in `backend/app/repositories/`:
   - `warehouse_repo.py` — CRUD for warehouses and inventory queries
   - `fleet_repo.py` — CRUD for vehicles and drivers
   - Each repository takes `AsyncSession` as constructor argument
4. **Create services** in `backend/app/services/`:
   - `warehouse_service.py` — Business logic for warehouse operations
   - `fleet_service.py` — Business logic for fleet operations
5. **Create API routes** in `backend/app/api/routes/`:
   - `warehouses.py` — GET /warehouses, GET /warehouses/{id}, PATCH /warehouses/{id} (toggle operational)
   - `vehicles.py` — GET /vehicles, GET /vehicles/{id}, PATCH /vehicles/{id} (toggle operational)
   - `health.py` — Move health check here
6. **Wire routes** into `main.py` using APIRouter

Verification

```
bash
# Run seed script
python scripts/seed_data.py

# Test API
curl http://localhost:8000/api/v1/warehouses
curl http://localhost:8000/api/v1/vehicles
curl http://localhost:8000/api/v1/warehouses/1/inventory
```

Phase 3: Order Intake & Fulfillment Group Creation

Goal: Accept new customer orders, run the warehouse-selection heuristic, split orders into fulfillment groups, and create inventory reservations — all in a single transactional flow.

What you'll understand: How multi-warehouse order splitting works, how the weighted heuristic scores warehouses, how inventory reservation prevents overselling, and how outbox events get written transactionally.

Tasks

1. **Create order schemas** in `backend/app/schemas/order.py`:
   - `OrderCreate` — customer_id, delivery_location (lat/lng), items list (sku, quantity)
   - `OrderRead`, `OrderDetail`, `FulfillmentGroupRead`
   - Status enums: `OrderStatus`, `FulfillmentGroupStatus`
2. **Create the Outbox writer** in `backend/app/events/outbox.py`:
   - `write_outbox_event(session, aggregate_type, aggregate_id, event_type, payload)` helper
   - Always called inside the same transaction as the state change
3. **Create** `backend/app/services/warehouse_selection.py` — The heuristic:
   - **Hard constraints**: warehouse must be operational, must have stock for requested SKUs
   - **Soft scoring**: weighted sum of normalized distance, current workload, estimated delivery time
   - **Weights**: configurable via settings (default: distance=0.4, workload=0.3, delivery_time=0.3)
   - **Order splitting**: if no single warehouse can fulfill everything, find the minimum set of warehouses that together cover all items
4. **Create** `backend/app/services/inventory_service.py`:
   - `reserve_inventory(session, warehouse_id, sku, quantity)` — creates Reservation row + increments `inventory.reserved`
   - `release_reservation(session, reservation_id)` — reverses the hold
   - All in the same transaction — no partial state
5. **Create** `backend/app/services/order_service.py`:
   - `create_order()` — the main flow:
     1. Validate items exist as SKUs
     2. Run warehouse selection heuristic
     3. Create FulfillmentGroups (one per warehouse)
     4. Reserve inventory for each group
     5. Set order status to `PENDING_ALLOCATION`
     6. Write outbox events for: order_created, fulfillment_group_created, inventory_reserved

- `cancel_order()` — reject if any group is OUT_FOR_DELIVERY+, otherwise cancel + release reservations

6. **Create API routes** `backend/app/api/routes/orders.py`:
   - POST /orders — create order
   - GET /orders — list orders with status filter
   - GET /orders/{id} — order detail with fulfillment groups
   - POST /orders/{id}/cancel — cancel order
7. **Create order repository** `backend/app/repositories/order_repo.py`

Verification

```
bash
# Create an order
curl -X POST http://localhost:8000/api/v1/orders \
  -H "Content-Type: application/json" \
  -d '{
    "customer_id": "...",
    "delivery_location": {"lat": 17.385, "lng": 78.486},
    "items": [{"sku": "SKU-001", "quantity": 2}, {"sku": "SKU-015", "quantity": 1}]
  }'

# Check the order was split into fulfillment groups
curl http://localhost:8000/api/v1/orders/{order_id}

# Verify inventory was reserved
curl http://localhost:8000/api/v1/warehouses/{warehouse_id}/inventory

# Check outbox events were written
# (query the database directly)
```

Phase 4: Batch Optimization Engine (OR-Tools CVRP)

Goal: Build the batch optimization loop that collects pending fulfillment groups, solves the Capacitated Vehicle Routing Problem using OR-Tools, and persists vehicle assignments and route stop sequences.

What you'll understand: How OR-Tools solves CVRP, how the distance matrix is built from coordinates, how batch triggering works (count or time threshold), and how the solver output maps to database routes and stops.

Tasks

1. **Create** `backend/app/optimization/distance.py`:
    - Build a distance matrix from coordinates (straight-line / Haversine)
    - Multiply by configurable speed factor to get estimated travel time
    - Clear documentation that this is an approximation (not road-network routing)
2. **Create** `backend/app/optimization/solver.py` — The OR-Tools CVRP wrapper:
    - Input: list of FulfillmentGroups with locations + weights, list of available Vehicles with capacities + depot locations
    - Build the OR-Tools routing model:
        - Distance callback from the matrix
        - Capacity dimension (weight_kg)
        - No time windows (v1)
    - Solve and return: `List[VehicleAssignment]` where each contains vehicle_id, ordered list of stops
    - Handle case where solver finds no feasible solution (some groups remain unassigned)
3. **Create** `backend/app/services/batch_optimizer.py` — The batch orchestrator:
    - Collect all fulfillment groups in `PENDING_ALLOCATION` status
    - Collect all operational, unassigned vehicles
    - If count ≥ 10 OR time since last solve ≥ 15 seconds → trigger solve
    - Call `solver.solve()`, then persist:
        - Create Route records (one per vehicle assignment)
        - Create RouteStop records (ordered stops)
        - Update FulfillmentGroup status to `ALLOCATED`
        - Update Order status (derived from group statuses)
        - Write outbox events: `route_created`, `fulfillment_group_allocated`
    - All in one transaction
4. **Create the batch trigger** — Background task in FastAPI:
    - Use `asyncio.create_task` in the lifespan to run a polling loop
    - Check conditions every 2 seconds, trigger solve when threshold met
    - Store `last_solve_time` in application state
5. **Create API routes** for optimization visibility:

- GET /routes — list active routes with stops
- GET /routes/{id} — route detail with stop sequence
- POST /optimization/trigger — manual trigger (for testing/demos)

6. **Write** `docs/algorithms.md` — Document the CVRP setup, distance approximation, and batch policy

Verification

```
bash
# Create 10+ orders to trigger batch solve
for i in $(seq 1 12); do
  curl -X POST http://localhost:8000/api/v1/orders \
    -H "Content-Type: application/json" \
    -d '{"customer_id": "...", "delivery_location": {"lat": 17.38, "lng": 78.48}, "items":
[{"sku": "SKU-001", "quantity": 1}]}'
done

# Wait for batch solve (or trigger manually)
curl -X POST http://localhost:8000/api/v1/optimization/trigger

# Check routes were created
curl http://localhost:8000/api/v1/routes

# Verify fulfillment groups are now ALLOCATED
curl http://localhost:8000/api/v1/orders
```

Phase 5: GPS Telemetry Simulator

Goal: Build the in-process simulator that moves vehicles along their assigned routes in compressed time, writes current positions to Redis, and persists periodic snapshots to PostgreSQL.

What you'll understand: How simulated time compression works, how linear interpolation along route stops creates smooth movement, how Redis serves as a fast-read cache for live position, and how PostgreSQL stores the audit trail.

Tasks

1. **Create** `backend/app/simulation/clock.py` — Simulated time manager:
   - Configurable compression ratio (e.g., 1 real second = 60 simulated seconds)
   - `get_simulated_time()` returns the current simulated timestamp
   - Track simulation start time and elapsed real time

2. **Create** `backend/app/simulation/interpolator.py` — Position calculator:
    - Given: ordered list of stop coordinates, vehicle speed (km/h), elapsed simulated time
    - Calculate: current position (lat/lng) by interpolating between stops
    - Detect: when a stop has been "reached" (within threshold distance)
    - Return: current segment, progress percentage, next stop ETA
3. **Create** `backend/app/simulation/engine.py` — The simulation loop:
    - For each active route with an assigned vehicle:
        - Calculate current interpolated position
        - Write to Redis: `vehicle:{id}:position → {lat, lng, timestamp, route_id, next_stop_index}`
        - Every N ticks, persist a TelemetrySnapshot row to PostgreSQL
        - When a stop is reached: update RouteStop status → ARRIVED → COMPLETED
        - When a stop is completed: update FulfillmentGroup status → OUT_FOR_DELIVERY → DELIVERED
        - When all stops done: update Route status → COMPLETED
        - Write outbox events for each state transition
    - Run as a background asyncio task, ticking every 1 real second
4. **Create Redis integration** `backend/app/core/redis.py`:
    - Async Redis client (using `redis.asyncio`)
    - Connection pool, typed read/write helpers
    - FastAPI dependency for route handlers
5. **Create API routes** for telemetry:
    - GET /vehicles/{id}/position — current position from Redis
    - GET /vehicles/{id}/telemetry — historical snapshots from PostgreSQL
    - GET /vehicles/positions — all current positions (batch read from Redis)
6. **Create simulation control endpoints**:
    - POST /simulation/start — begin simulation
    - POST /simulation/stop — pause simulation
    - GET /simulation/status — current simulated time, active vehicles count

Verification

```
bash
# Trigger an optimization run first (to create routes)
curl -X POST http://localhost:8000/api/v1/optimization/trigger

# Start simulation
curl -X POST http://localhost:8000/api/v1/simulation/start

# Watch vehicle positions update
watch -n 1 'curl -s http://localhost:8000/api/v1/vehicles/positions | python -m json.tool'

# Check telemetry history
curl http://localhost:8000/api/v1/vehicles/{id}/telemetry
```

Phase 6: Disruption Handling (Vehicle Breakdown & Warehouse Outage)

Goal: Implement first-class failure scenarios — vehicle breakdowns cause orphaned stops to be re-collected and re-optimized; warehouse outages block new allocations and allow group reassignment.

What you'll understand: How disruption detection works, how orphaned fulfillment groups are pooled for re-solve, how the visible delay during recalculation is intentional (not a bug), and how assignment history maintains auditability.

Tasks

1. **Create** `backend/app/services/disruption_service.py`:
    - `handle_vehicle_breakdown(vehicle_id)`:
        1. Mark vehicle as non-operational
        2. Stop simulation for this vehicle
        3. Collect all incomplete RouteStops from the vehicle's active route
        4. Reset affected FulfillmentGroups to `PENDING_ALLOCATION`
        5. Release any specific vehicle lock on those groups
        6. Record assignment history (what was re-assigned and why)
        7. Write outbox events: `vehicle_breakdown`, `fulfillment_groups_orphaned`
        8. Trigger a re-solve for the orphaned sub-batch
    - `handle_warehouse_outage(warehouse_id)`:
        1. Mark warehouse as non-operational

2. Block new fulfillment group creation for this warehouse
3. Any groups in PENDING_ALLOCATION at this warehouse: hold them
4. Groups already OUT_FOR_DELIVERY: unaffected (inventory already picked)
5. Write outbox events: `warehouse_outage`

- `restore_warehouse(warehouse_id)`:
  1. Mark warehouse as operational
  2. Release held fulfillment groups back to the allocation queue
  3. Write outbox event: `warehouse_restored`

2. **Create assignment history table/model** — `route_assignment_history`:
   - Records: fulfillment_group_id, old_vehicle_id, new_vehicle_id, reason, timestamp
   - Queryable for audit
3. **Extend batch optimizer** for re-solve:
   - Accept a sub-batch of specific fulfillment groups (not just pending ones)
   - Use only remaining operational vehicles
   - The re-solve should be visible in the UI as a "recalculating" state
4. **API routes** for disruption:
   - POST /vehicles/{id}/breakdown — trigger vehicle breakdown
   - POST /vehicles/{id}/restore — restore vehicle to operational
   - POST /warehouses/{id}/outage — trigger warehouse outage
   - POST /warehouses/{id}/restore — restore warehouse
   - GET /disruptions — list active disruptions and affected groups

Verification

```
bash
# Start with routes running and simulation active
# Trigger a breakdown
curl -X POST http://localhost:8000/api/v1/vehicles/{id}/breakdown

# Watch the affected fulfillment groups get re-assigned
curl http://localhost:8000/api/v1/routes

# Check assignment history
curl http://localhost:8000/api/v1/disruptions

# Trigger a warehouse outage
curl -X POST http://localhost:8000/api/v1/warehouses/{id}/outage

# Verify no new groups get assigned to this warehouse
curl -X POST http://localhost:8000/api/v1/orders -d '...'
```

Phase 7: Frontend Setup & Dashboard Shell

Goal: Initialize the React + TypeScript + Vite project, set up Tailwind CSS, create the layout shell, and build the main dashboard page with live data from the API.

What you'll understand: How the frontend project is structured, how API data flows into React components, how Tailwind creates the operational UI density, and how the layout provides consistent navigation.

Tasks

1. **Initialize frontend project**:
    - `npm create vite@latest` with React + TypeScript template
    - Install: `tailwindcss`, `@tailwindcss/vite`, `react-router-dom`, `axios`, `recharts`, `leaflet`, `react-leaflet`, `@types/leaflet`
    - Configure `vite.config.ts` with API proxy to backend
2. **Create** `frontend/src/services/api.ts` — Axios client with base URL
3. **Create TypeScript types** in `frontend/src/types/`:
    - `order.ts`, `warehouse.ts`, `vehicle.ts`, `route.ts`, `common.ts`
    - Mirror the backend Pydantic response schemas

4. **Create layout** `frontend/src/layouts/DashboardLayout.tsx`:
    - Sidebar navigation: Dashboard, Orders, Fleet, Warehouses, Routes, Disruptions
    - Top bar: simulation status indicator, current simulated time
    - Main content area
5. **Create dashboard page** `frontend/src/pages/Dashboard.tsx`:
    - **Summary cards**: Total orders (by status), Active vehicles, Active routes, Disruptions count
    - **Recent orders table**: Last 20 orders with status badges
    - **Warehouse status panel**: Each warehouse with operational/outage indicator
    - Auto-refresh every 5 seconds using polling
6. **Create reusable components**:
    - `StatusBadge.tsx` — Color-coded status indicator
    - `DataTable.tsx` — Generic sortable/filterable table
    - `SummaryCard.tsx` — Metric card with icon and value
    - `LoadingSpinner.tsx`, `ErrorBanner.tsx`
7. **Set up React Router** with routes for all planned pages

Verification

```
bash
cd frontend
npm install
npm run dev
# Open http://localhost:5173
# Dashboard should show live data from the backend API
```

Phase 8: Map View & Vehicle Tracking UI

Goal: Build the interactive Leaflet map showing warehouse locations, vehicle positions (updating live), route paths, and delivery stops. This is the centerpiece of the operator UI.

What you'll understand: How Leaflet integrates with React, how live vehicle positions are polled and rendered, how route polylines are drawn, and how clicking a vehicle shows its assigned route and stops.

Tasks

1. **Create** `frontend/src/components/Map/`:
   - `MapContainer.tsx` — Leaflet map centered on Hyderabad with OSM tiles
   - `WarehouseMarker.tsx` — Warehouse icons (green=operational, red=outage)
   - `VehicleMarker.tsx` — Vehicle icons that move with position updates, with heading indicator
   - `RoutePolyline.tsx` — Draw the route path as a colored line
   - `StopMarker.tsx` — Delivery stop markers (gray=pending, blue=in-transit, green=completed)
   - `MapLegend.tsx` — Legend explaining marker colors
2. **Create hooks**:
   - `useVehiclePositions()` — Polls GET /vehicles/positions every 2 seconds
   - `useWarehouses()` — Fetch warehouse list
   - `useRoutes()` — Fetch active routes with stops
   - `useSimulationStatus()` — Fetch current simulation state
3. **Create Fleet Tracking page** `frontend/src/pages/FleetTracking.tsx`:
   - Full-width map showing all warehouses and vehicles
   - Click vehicle → sidebar with: vehicle details, assigned route, stop list with statuses
   - Click warehouse → sidebar with: warehouse info, inventory summary, assigned vehicles
4. **Create Vehicle Detail page** `frontend/src/pages/VehicleDetail.tsx`:
   - Vehicle info card
   - Current route with stop progress
   - Telemetry history chart (position over time)

Verification

```
bash
# With simulation running, open the Fleet Tracking page
# Vehicles should visibly move on the map
# Click a vehicle → see its route and stops
# Click a warehouse → see its inventory and status
```

Phase 9: Orders & Operations Management UI

Goal: Build the full order management interface — order list, order detail, create order form, and disruption management panels. Complete the operator workflow.

What you'll understand: How the complete operator workflow flows through the UI, how order status changes propagate visually, and how disruption triggers are surfaced.

Tasks

1. **Create Orders page** `frontend/src/pages/Orders.tsx`:
    - Filterable table: status filter, date range, search by order ID
    - Status breakdown chart (Recharts bar/pie)
    - Click row → navigate to order detail
2. **Create Order Detail page** `frontend/src/pages/OrderDetail.tsx`:
    - Order header: ID, customer, status, timestamps
    - Fulfillment groups list: each with warehouse, items, status, assigned vehicle
    - Cancel button (disabled if any group is OUT_FOR_DELIVERY+)
    - Mini-map showing delivery location and route
3. **Create New Order form** `frontend/src/pages/CreateOrder.tsx`:
    - Customer selection
    - Item picker with SKU search, quantity input
    - Delivery location picker (click on map or enter coordinates)
    - Submit → shows confirmation with warehouse allocation result
4. **Create Warehouses page** `frontend/src/pages/Warehouses.tsx`:
    - Warehouse cards with: name, location, status, inventory summary
    - Toggle operational status
    - Outage/restore buttons
5. **Create Disruptions page** `frontend/src/pages/Disruptions.tsx`:
    - Active disruptions panel
    - Vehicle breakdown: trigger button, affected groups list, re-solve status
    - Warehouse outage: trigger button, held groups list
    - Assignment history timeline
6. **Create Simulation Control panel** (in sidebar or separate page):

- Start/Stop simulation
- Current simulated time display
- Speed control

Verification

```
bash
# Full end-to-end workflow:
# 1. Create orders via the UI
# 2. Watch them get allocated (batch optimization)
# 3. Start simulation → watch vehicles move on map
# 4. Trigger a breakdown → watch re-optimization happen
# 5. See orders progress to DELIVERED
```

Phase 10: Testing, Polish & Documentation

Goal: Write unit and integration tests for critical business logic, fix edge cases, finalize documentation, and ensure the full demo workflow is smooth end-to-end.

What you'll understand: How to test transactional business logic, how to verify the optimization produces correct results, and how to document the system for a design review.

Tasks

1. **Unit tests** (`backend/tests/unit/`):
   - `test_warehouse_selection.py` — Heuristic scoring, hard constraints, order splitting
   - `test_inventory_service.py` — Reserve, release, concurrent reservation safety
   - `test_order_service.py` — Order creation flow, status derivation, cancellation rules
   - `test_solver.py` — OR-Tools wrapper with known inputs → verify feasible output
   - `test_interpolator.py` — Position interpolation along route segments
   - `test_disruption.py` — Breakdown handling, group orphaning, re-solve triggering
2. **Integration tests** (`backend/tests/integration/`):
   - `test_order_flow.py` — Full order → allocation → delivery flow against a test database
   - `test_outbox.py` — Verify outbox events are written transactionally
   - `test_batch_optimization.py` — End-to-end batch solve with real OR-Tools
3. **Finalize documentation**:

- Complete `docs/database.md` — Final ERD with all columns
- Complete `docs/api.md` — All endpoints with request/response examples
- Complete `docs/events.md` — All outbox event types with payload schemas
- Complete `docs/algorithms.md` — CVRP setup, heuristic scoring, simulation details
- Update `README.md` — Setup instructions, architecture overview, demo walkthrough

4. **Polish**:
   - Error handling on all API endpoints (proper HTTP status codes)
   - Loading states and error boundaries in frontend
   - Consistent API response format
   - Input validation for all endpoints
5. **Create demo script** `scripts/run_demo.sh`:
   - Starts infrastructure
   - Runs migrations
   - Seeds data
   - Starts backend
   - Starts frontend
   - Creates sample orders
   - Starts simulation

Verification

```
bash
# Run all tests
cd backend && pytest -v --cov=app

# Run the full demo
./scripts/run_demo.sh

# Walkthrough all UI pages and verify everything works
```

Phase Dependencies

```
```

> **TIP**
>
> **Phases 7-8 (frontend) can start in parallel with Phases 5-6 (backend)** once Phase 4 is done. This lets you see the system visually while still building the simulation and disruption handling.

Tech Stack Summary

Layer	Technology	Purpose
Backend Framework	FastAPI + Uvicorn	Async HTTP API
ORM	SQLAlchemy 2.0 (async)	Database models & queries
Database	PostgreSQL 16 + PostGIS	Durable state + geospatial
Migrations	Alembic	Schema versioning
Cache	Redis	Live vehicle positions
Optimization	Google OR-Tools	CVRP batch solver
Frontend	React 18 + TypeScript	Operator UI
Build Tool	Vite	Frontend bundling
Styling	Tailwind CSS	Utility-first CSS
Maps	Leaflet + react-leaflet + OSM	Interactive map
Charts	Recharts	Dashboard visualizations
Testing	Pytest + pytest-asyncio	Backend tests
Containers	Docker Compose	Local dev infrastructure

File Structure (Final)

```
RouteGrid/
├── backend/
│   ├── pyproject.toml
│   ├── alembic.ini
│   ├── app/
│   │   ├── main.py                    # FastAPI app factory
│   │   ├── __init__.py
│   │   ├── api/
│   │   │   ├── __init__.py
│   │   │   └── routes/
│   │   │       ├── health.py
│   │   │       ├── orders.py
│   │   │       ├── warehouses.py
│   │   │       ├── vehicles.py
│   │   │       ├── routes.py
│   │   │       ├── simulation.py
│   │   │       └── disruptions.py
│   │   ├── core/
│   │   │   ├── config.py              # Pydantic Settings
│   │   │   ├── auth.py                # get_current_user seam
│   │   │   └── redis.py               # Redis client
│   │   ├── db/
│   │   │   ├── engine.py              # Async engine
│   │   │   ├── session.py             # Async session + FastAPI dep
│   │   │   └── base.py                # Declarative base
│   │   ├── models/
│   │   │   ├── order.py
│   │   │   ├── fulfillment.py
│   │   │   ├── warehouse.py
│   │   │   ├── reservation.py
│   │   │   ├── fleet.py
│   │   │   ├── route.py
│   │   │   ├── telemetry.py
│   │   │   └── outbox.py
│   │   ├── schemas/
│   │   │   ├── order.py
│   │   │   ├── warehouse.py
│   │   │   ├── fleet.py
│   │   │   ├── inventory.py
│   │   │   ├── route.py
│   │   │   └── common.py
│   │   ├── repositories/
│   │   │   ├── order_repo.py
```

```
│   │   │   ├── warehouse_repo.py
│   │   │   ├── fleet_repo.py
│   │   │   └── route_repo.py
│   │   ├── services/
│   │   │   ├── order_service.py
│   │   │   ├── warehouse_service.py
│   │   │   ├── warehouse_selection.py
│   │   │   ├── inventory_service.py
│   │   │   ├── fleet_service.py
│   │   │   ├── batch_optimizer.py
│   │   │   └── disruption_service.py
│   │   ├── optimization/
│   │   │   ├── distance.py
│   │   │   └── solver.py
│   │   ├── simulation/
│   │   │   ├── clock.py
│   │   │   ├── interpolator.py
│   │   │   └── engine.py
│   │   └── events/
│   │       └── outbox.py
│   ├── migrations/
│   │   └── versions/
│   └── tests/
│       ├── unit/
│       └── integration/
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.js
│   └── src/
│       ├── App.tsx
│       ├── main.tsx
│       ├── components/
│       │   ├── Map/
│       │   ├── StatusBadge.tsx
│       │   ├── DataTable.tsx
│       │   └── SummaryCard.tsx
│       ├── pages/
│       │   ├── Dashboard.tsx
│       │   ├── FleetTracking.tsx
│       │   ├── Orders.tsx
│       │   ├── OrderDetail.tsx
│       │   ├── CreateOrder.tsx
│       │   ├── Warehouses.tsx
│       │   ├── VehicleDetail.tsx
│       │   └── Disruptions.tsx
```

```
│       ├── layouts/
│       │   └── DashboardLayout.tsx
│       ├── hooks/
│       │   ├── useVehiclePositions.ts
│       │   ├── useOrders.ts
│       │   └── useSimulationStatus.ts
│       ├── services/
│       │   └── api.ts
│       ├── types/
│       │   ├── order.ts
│       │   ├── warehouse.ts
│       │   └── vehicle.ts
│       └── utils/
├── data/
│   └── seed/
├── docs/
│   ├── requirements.md
│   ├── architecture.md
│   ├── database.md
│   ├── api.md
│   ├── events.md
│   └── algorithms.md
├── scripts/
│   ├── seed_data.py
│   └── run_demo.sh
├── docker-compose.yml
├── .env.example
├── .gitignore
└── README.md
```

> **NOTE**
>
> This plan is designed so you can execute it **phase by phase**, with clear verification commands after each phase. Each phase produces working, testable code — you never have to build blindly and hope it works at the end.